

Arquitetura, camadas e fluxo de implementação

Um guia visual do que já foi construído, por que cada camada existe e como evoluir o monólito modular com segurança.

Recorte deste documento: estado do código em 12 de setembro de 2026. O código e os testes são a fonte principal; o README ainda é parcial para a fatia de ExamDefinition.

IDEIA CENTRAL

Aprender por fatia vertical: entidade -> repositório -> serviço -> controller, mantendo cada responsabilidade no seu lugar.

Vinícius Oliveira | Java 21 + Spring Boot 4.1.1

1. Visão do projeto

O MedFlow é um sistema de gestão de fluxo de exames. A escolha atual é um **monólito modular**: uma aplicação Spring Boot, um repositório e módulos de negócio separados por pacote. É uma forma adequada de consolidar domínio, API, JPA e testes antes de introduzir a complexidade operacional de microserviços.

Monólito modular não é um microserviço disfarçado. Hoje os módulos se chamam diretamente dentro do mesmo processo. A separação por domínio cria fronteiras de código que, se fizer sentido no futuro, podem orientar uma extração gradual.

Monolito modular - desenho atual

Módulos de negócio organizados em fatias verticais; ainda não são serviços distribuídos.

patient

domínio do paciente
application | persistence | web

exam.order

pedido de exame
application | persistence | web

exam.definition

catálogo de exames
application | persistence | web

shared

erros HTTP e preocupações comuns
sem regra de negócio específica

Padrão repetido dentro de cada módulo

domain -> persistence -> application -> web

O pacote **shared** é reservado para preocupações realmente comuns, como a padronização de erros HTTP. Regra de negócio específica não deve ir para ele apenas para evitar repetição.

Módulos já iniciados

Módulo	Responsabilidade atual	Situação
patient	Cadastro e manutenção de pacientes.	Fatia vertical já existente.
exam.order	Pedido de exames vinculado ao paciente.	Fatia existente; há reorganização local de DTOs em andamento.
exam.definition	Catálogo de tipos de exame: código, nome, amostra e status.	Entidade, repository, service, controller e testes implementados.
shared.web.error	Respostas de erro em formato ProblemDetail.	Já atende erros de paciente/pedido e validações gerais.

2. O papel de cada camada

A divisão não é burocracia: cada camada protege uma decisão diferente. Quando uma regra está no lugar certo, o controller permanece fino, o serviço concentra o caso de uso e a entidade protege o domínio mesmo fora da API.

Camada	Pergunta que responde	Exemplos no MedFlow
domain	O objeto pode existir neste estado?	ExamDefinition, SampleType, validações do construtor, update e deactivate.
persistence	Como consultar ou gravar?	ExamDefinitionRepository e os mapeamentos JPA.
application	Como executar o caso de uso?	ExamDefinitionService; DTOs de entrada e de saída; exceções de negócio.
web	Como expor o caso de uso em HTTP?	ExamDefinitionController, @Valid, status HTTP e paginação.
shared	Qual regra técnica é comum a vários módulos?	GlobalExceptionHandler, ProblemTypes e estrutura de erro.

Validação em três fronteiras

Onde	O que protege	Exemplo
DTO de entrada	Formato do HTTP recebido antes do caso de uso.	@NotBlank, @Size e @Valid no controller.
Entidade	Invariantes do domínio em qualquer ponto de criação/alteração.	Código do exame normalizado; nome, sampleType e limites verificados.
Coluna JPA	Contrato de persistência quando houver banco configurado.	unique, nullable, length e @Version no mapeamento.

As validações se complementam. A anotação no DTO melhora a resposta para o cliente; a entidade impede estado inválido por qualquer entrada; a coluna protege o armazenamento. Uma não substitui a outra.

3. Fluxo de uma requisição

O caminho abaixo usa a criação de uma definição de exame. A mesma ideia organiza as outras fatias: a borda adapta HTTP; o serviço orquestra; o domínio valida seu estado; o repository representa a persistência; o DTO de resposta evita expor a entidade.

```
POST /exam-definitions
JSON -> @Valid no Controller
      -> ExamDefinitionService.create(request)
      -> new ExamDefinition(code, name, sampleType)
      -> repository.existsByCode(code) / repository.save(entity)
      -> ExamDefinitionResponseDto
      -> HTTP 201 Created
```

Etapa	Responsabilidade	Por que importa
Controller	Recebe o contrato HTTP e delega.	Não deve conter regra de negócio ou acesso direto ao repository.
Service	Cria a entidade, checa código duplicado, salva e monta a resposta.	É o local visível do caso de uso e das regras que envolvem mais de um objeto.
Entity	Normaliza e valida dados no construtor.	Mesmo que alguém crie o objeto fora da API, a regra continua protegida.
Repository	Oferece existsByCode, save, findById e findAll.	Evita espalhar detalhes de JPA pela aplicação.
Response DTO	Expõe apenas dados definidos no contrato de saída.	Evita acoplamento entre API e entidade de persistência.

Erro e resposta

Erros HTTP são padronizados no **GlobalExceptionHandler** com ProblemDetail. Já há resposta consistente para validação (422), JSON inválido (400), paciente não encontrado e pedido não encontrado (404). Para a fatia de ExamDefinition, as classes de exceção já existem; o mapeamento específico de **não encontrado (404)** e **código duplicado (409)** ainda é o próximo ajuste de integração.

4. Fatia vertical: ExamDefinition

ExamDefinition representa o catálogo de exames que podem ser solicitados. Ele não é o pedido de um paciente: é a definição reutilizável de um tipo de exame, por exemplo, um hemograma com código próprio e tipo de amostra.

Elemento	Decisão implementada	Intenção
code	Obrigatório, único, até 50 caracteres, convertido para maiúsculas com trim.	Tornar o identificador estável; ' hem01 ' e 'HEM01' representam o mesmo código.
name	Obrigatório, até 150 caracteres, com trim.	Evitar rótulos vazios e espaços acidentais.
sampleType	Enum obrigatório: BLOOD, URINE, STOOL, SWAB ou OTHER.	Evitar texto livre para uma classificação controlada.
active	Começa ativo; deactivate altera o estado sem apagar o registro.	Preserva histórico e possibilita soft delete.
timestamps e version	createdAt, updatedAt e @Version presentes na entidade.	Preparam auditoria e concorrência otimista quando JPA/banco estiverem configurados.

Contratos de aplicação

Objeto	Uso
CreateExamDefinitionRequestDto	Entrada de criação: code, name e sampleType, com validação de borda.
UpdateExamDefinitionRequestDto	Entrada de alteração: name e sampleType. O código não é alterado.
ExamDefinitionResponseDto	Saída formatada da API a partir da entidade.
ExamDefinitionNotFoundException	Expressa que o id consultado/alterado não existe.
ExamDefinitionCodeConflictException	Expressa conflito de unicidade quando o código já existe.

A service também possui **deactivate(id)**, mas não há rota HTTP para ela por decisão de escopo: a API atual possui apenas leitura, criação e atualização. Isso evita introduzir PATCH antes de precisar de um contrato claro para a operação.

5. API e verificação

A API de definições de exames foi desenhada com quatro operações simples e consistentes com os módulos já existentes.

Método e rota	Caso de uso	Resposta normal
GET /exam-definitions	Lista definições, com paginação Spring.	200 OK + Page
GET /exam-definitions/{id}	Busca uma definição pelo identificador.	200 OK + DTO
POST /exam-definitions	Cria uma definição nova.	201 Created + DTO
PUT /exam-definitions/{id}	Atualiza nome e tipo de amostra.	200 OK + DTO

Como os testes foram pensados

Nível	Foco	Por que esse tipo de teste
Domínio	Construtor, normalização, update, deactivate e regras de validade.	Confirma as invariantes sem depender de Spring ou banco.
Service	Orquestração, conflito de código, busca e atualização usando repository mockado.	Testa caso de uso de modo rápido e isolado.
DTO	Anotações de Bean Validation nas entradas e construção da resposta.	Garante que o contrato da borda rejeite dados inválidos.
Controller	Rotas, status, delegação, paginação e POST inválido com 422 via MockMvc.	Valida a integração da camada web sem iniciar infraestrutura externa.

Última validação registrada: a suíte completa Maven executou 78 testes, com 0 falhas e 0 erros, antes do commit do controller em 12/09/2026. Isso confirma o comportamento coberto pelos testes; não confirma infraestrutura ainda não configurada.

Convenção de teste importante: teste não é só uma etapa final. Ele registra o comportamento esperado de uma camada e permite alterar o projeto com segurança depois.

6. Decisões e evolução arquitetural

O projeto tem dependências e intenção de evoluir para integrações distribuídas, mas a arquitetura atual conscientemente mantém essa complexidade fora da fatia de domínio. A ordem de aprendizado é: consolidar casos de uso e fronteiras; depois integrar; por último decidir se extrair serviços vale o custo.

Tema	Estado atual	Evolução consciente
Banco e migrations	Entidades e repositories JPA existem; datasource e migrations reais não foram validados.	Configurar PostgreSQL/Flyway e testar a persistência de verdade.
Spring Security	A dependência está presente; a aplicação ainda usa a senha automática de desenvolvimento.	Definir autenticação, autorização, usuários/roles e regras por rota.
Mensageria	RabbitMQ não participa das operações atuais.	Publicar eventos de domínio quando existir um consumidor real e contrato definido.
Comunicação síncrona	Não há OpenFeign em uso dentro do monólito.	Só faz sentido após a extração de uma fronteira para outro processo.
API Gateway	Não existe gateway em execução.	Introduzir ao expor múltiplos serviços externos, não como passagem interna do monólito.

Quando usar comunicação interna e assíncrona?

Situação	Escolha adequada	Exemplo futuro
Mesmo monólito	Chamada direta entre services, respeitando interfaces e domínio.	ExamOrder consulta uma definição por uma porta de aplicação, sem HTTP para si mesmo.
Dado necessário na resposta imediata entre serviços	HTTP/OpenFeign depois de extrair um serviço.	Order Service precisa validar uma definição mantida em Laboratory Service.
Fato que outros módulos podem consumir sem bloquear o fluxo	Evento assíncrono em RabbitMQ/Kafka.	Pedido criado -> Notification Service envia uma notificação.

Regra prática: não criar mensageria ou OpenFeign apenas porque são tecnologias desejadas. Primeiro defina o fato de negócio, o dono do dado, o consumidor e a necessidade de consistência/tempo de resposta.

7. Mapa de estado e próximo passo

Este quadro evita confundir intenção arquitetural com capacidade disponível. Ele é útil para decidir o próximo código sem antecipar infraestrutura que ainda não tem caso de uso definido.

Já construído	Pendente de integração	Planejado para evolução
Fatias patient, exam.order e exam.definition.	Tratador global específico para ExamDefinition: 404 e 409.	PostgreSQL, Flyway e migrations novas.
Entidades com regras e mapeamentos JPA.	Endpoint de desativação, se o contrato HTTP for definido.	Spring Security com autenticação e autorização.
Repositories, services, DTOs e controllers de ExamDefinition.	Persistência real e testes de integração com banco.	Eventos assíncronos, contratos e consumidores.
ProblemDetail para erros já conectados ao handler.	Atualização do README para incluir ExamDefinition.	Extração gradual de módulos para serviços, se justificada.
Testes de domínio, service, DTO e web; 78 testes passaram na última execução registrada.	Padronizar a reorganização de DTOs que está em andamento em exam.order.	OpenFeign e API Gateway somente após fronteiras externas reais.

Próxima sequência recomendada

Passo	Objetivo
1. Fechar erros de ExamDefinition	Adicionar os tipos e handlers 404/409, com testes de web. Fecha a fatia já exposta.
2. Definir persistência real	Escolher a configuração local de PostgreSQL e criar migrations novas, sem alterar migrations aplicadas.
3. Modelar segurança	Definir quem são os usuários, quais perfis existem e quais operações cada perfil pode realizar.
4. Avançar no próximo caso de uso	Repetir o fluxo entidade -> repository -> service -> controller, sempre explicando o papel de cada decisão.

Resumo: o MedFlow já tem uma base modular concreta. O próximo ganho não é adicionar mais tecnologia; é completar o contrato de erro da fatia atual e consolidar a persistência real. A partir daí, as escolhas de segurança e integração terão um domínio mais estável para apoiar.

Documento de apoio ao desenvolvimento - não substitui o código, os testes e as decisões registradas no repositório.